

Issues_Summary

Open workable items (current_location = Issues), regenerated from the SQLite single-source-of-truth (§11.4.12).

Counts by Type × Status

Type	Status	Count
Bug	Operator-blocked	1
Bug	Queued	11
Task	In progress	3
Task	Operator-blocked	3
Task	Queued	34
TOTAL		52

Items

ATM ID	Type	Status	Severity	Description
BOB-008	Bug	Operator-blocked	—	RuTracker automated login blocked by CAPTCHA
BOB-065	Task	Queued	High	Lava P2: Egress diagnosis and VPN-host SOCKS routing (containers pkg/egress)
BOB-066	Task	In progress	High	Lava P3: BOBA_UPSTREAM_PROXY in download-proxy + qBitTorrent-go + Jac compose env-forward
BOB-068	Bug	Queued	High	RD2-00: unattributed, unreviewed Auto mechanism pushing to main
BOB-069	Task	Queued	High	RD2-40: §11.4.238 QA-discovery-channel ledger — ongoing coverage-escape back
BOB-070	Bug	Queued	High	RD2-41: pre-build mutation-marker scan carrier false-positive silently defeats en build gate
BOB-074	Task	Queued	Medium	RD2-07: DDoS-class testing fully absent the mandated test-type matrix
BOB-077	Task	Operator-blocked	High	RD2-10: Identify second host running the commit rsync/sync mechanism (OPERAT DECISION)

ATM ID	Type	Status	Severity	Description
BOB-078	Task	Queued	High	RD2-11: Once identified, wire Auto-commit mechanism through §11.4.234 dedicated commit/push script OR retire it
BOB-079	Task	Queued	Medium	RD2-12: Retroactive attributed history n for GA-18/21/22/25/26/27 changes (never rewrite published history)
BOB-080	Task	Queued	High	RD2-13: Retroactive Fable-xhigh code n of the substantive Auto-commit diffs
BOB-081	Task	Queued	High	RD2-14: Author CONTINUATION.md Section 15 entry (currently 53 days / 24+ commits behind HEAD)
BOB-082	Task	Queued	High	RD2-15: Create BOB-064..067 workable for the four Lava-porting findings (close GA-05)
BOB-083	Task	Queued	Medium	RD2-16: Regenerate browser_extension features/codegraph Status.md + Summary HTML/PDF siblings
BOB-084	Task	Queued	High	RD2-17: Reconcile BOB-008 DB/MD books via the workable-items tool
BOB-085	Task	Queued	Medium	RD2-18: Create top-level Boba proxy/middleware service v1.0.0 readiness ledger (closes 0)
BOB-086	Task	Queued	Medium	RD2-19: Fix BOB-009/BOB-010 evidence + backfill item_history for 56 silent closes
BOB-087	Task	Queued	High	RD2-20: Wire docs_chain / commit-seam hook per §11.4.106(F) so DB writes can land without MD mirror
BOB-088	Task	Queued	Medium	RD2-21: Complete/verify README Tracker Items + Status Documents table row-completeness (GA-07 remainder)
BOB-089	Task	Queued	High	RD2-24: RED-first tests for start.sh reload_python/reload_plugins/recreate_ (closes test-half of GA-27)
BOB-090	Task	Queued	Medium	RD2-25: HelixQA Challenge entry exercise three start.sh subcommands end-to-end real compose stack
BOB-092	Bug	Queued	Medium	RD2-27: Remove test_live_stack_evidence.py:265 nnmcl on-404 fallback + verify live 200 (closes 0)
BOB-093	Task	Queued	High	RD2-28: Live compose bring-up + verify rustracker ReDoS regex bounds deployed container (closes runtime half of GA-12)
BOB-094	Task	Queued	Medium	RD2-29: Author tests/stress/test_tracker_fetch_stress_chaos.py with §11.4.85 fault injection
BOB-095	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				RD2-30: Author tests/stress/test_scheduler_hooks_sse_stress_chaos. Go-side triangle
BOB-096	Task	Queued	Medium	RD2-31: Extend qBitTorrent-go jackett_db_test.go with real process-kill resource-exhaustion fault injection
BOB-097	Task	Queued	Low	RD2-32: Author DDoS-class coverage for exposed download-proxy/merge endpoints (canonical impl of RD2-07)
BOB-098	Task	Queued	Medium	RD2-34: Parametrize 20 hardcoded /VolT7 paths in helixqa banks with PROJECT (closes GA-23)
BOB-099	Bug	Queued	Medium	RD2-36: Fix guard-forbidden-commands substring-match false-positive class + a const033-poweroff-signal-triage carrier EXCLUDE_PATHS
BOB-100	Task	Queued	Low	RD2-39: Bump submodules/jackett one (canonical impl of RD2-09)
BOB-101	Task	Operator-blocked	High	GA-19/RW-09: Is -profile go parity still a release goal? (gates RW-10..13) — OPERATOR-DECISION
BOB-102	Task	Operator-blocked	Medium	RW-05: LAN-exposure threat model — b tunnel 127.0.0.1 or keep 0.0.0.0? — OPERATOR-DECISION
BOB-104	Task	In progress	Medium	§11.4.238 followup: CodeGraph 1.5.0 nested-.gitignore regression challenge
BOB-105	Task	Queued	Medium	§11.4.238 followup: mechanical §11.4.2 anchor-block-integrity check
BOB-106	Task	Queued	Medium	§11.4.238 followup: §11.4.84 quiescence helper for the unattributed auto-commit
BOB-107	Task	Queued	Medium	§11.4.238 followup: pre-dispatch existence check for subagent task-brief source in
BOB-109	Task	Queued	Medium	BOB-074 followup: scaling-class test coverage absent from mandated test-type matrix
BOB-110	Task	Queued	Medium	BOB-074 followup: UX-class test coverage (accessibility/usability) absent
BOB-111	Task	In progress	High	BOB-074 followup: configure real rate limit for boba's 3 public HTTP endpoints
BOB-113	Task	Queued	Low	BOB-074 followup: add wrk to dev tooling DDoS/load challenges
BOB-114	Task	Queued	Medium	BOB-074 followup: self-validation golden fixture for the rate-limit detector
BOB-118	Bug	Queued	High	

ATM ID	Type	Status	Severity	Description
				README.md python-tests badge claims passing; pytest -collect-only measures 5 stale/wrong)
BOB-120	Bug	Queued	Critical	3rd forced-logout incident 2026-08-18 2 — SIGKILL user@1000 + preventive-ti inside-user-slice architectural gap
BOB-121	Task	Queued	Important	External watchdog for the forced-logout architectural gap (task #85, incident #3
BOB-129	Bug	Queued	Medium	Potential production slowapi/starlette d flagged by Task 105 subagent
BOB-131	Bug	Queued	Medium	qbittorrent-proxy podman common crash pre-existing, surfaced during BOB-129 investigation
BOB-135	Bug	Queued	Low	Test isolation: test_list_hooks_after_crea in bulk suite (Permission denied /config
				WHAT. A single 2026-08-20 sweep found workable-item rows whose tracked state contradicted reality: BOB-076 (closed b commit 99a486e on 2026-08-15, whose message literally says “closes BOB-076 Queued), BOB-117 (fix landed AND cover a permanent guard assertion, still Queued BOB-091 (implemented under d1a8479 a3b16bc / 2a0b543, still Queued), and BOB-008 whose Evidence field still quoted diagnostic string that BOB-117 removed source. CORRECTION 2026-08-20 (11.4 earlier revision of this item claimed the enabling defect was that “workable-item is blind to body_md drift”. That was WR and is corrected here rather than quietly edited. The engine is NOT blind. The FLAGLESS INVOCATION is: workable-i diff -db docs/workable_items.db -> “diff and Markdown are in sync” (opens ZERO files) workable-items diff -db docs/workable_items.db -issues docs/Issues.r -fixed docs/Fixed.md -> “~ BOB-008 bo differs (md=1346 bytes db=1333 bytes, “diff: 1 difference(s)” Both measured on same seeded body-only drift. An independent agent proved the mechanism with strack flagless form never opens any Markdown So this is a 11.4.201(6) FALSE-NULL of CALLER, not a gap in the comparison logic. Importantly, pre_build_verification.sh in 17 already passes both flags, so the STAGATE was never blind — the drift above
BOB-136	Task	Queued	High	

ATM ID	Type	Status	Severity	Description
				not caused by a blind gate. THE REAL DEFECTS, restated honestly. (1) The flag form is a false-null trap: it reports a commit as reassuring “in sync” while checking not. Any caller using it is a green-reporting check. It should either default to the conventional Issues.md/Fixed.md paths REFUSE with a clear error, never return a clean verdict for a comparison it did not perform. (2) Work commonly lands under commit messages that do not carry the (GA-14/15/16 for BOB-091), so nothing mechanically links a commit back to its and closure depends on a human remembering. 11.4.227: an anchor done is its SEAM landing, not its TEXT landing ACCEPTANCE. (a) The flagless diff invocation no longer returns a clean verdict without reading Markdown, with a paired 1.1 m proving it. (b) A repo-wide grep confirms caller uses the flagless form. (c) A swee flags any non-terminal item whose id appears in a merged commit message, so done-h open rows are found mechanically rather by a human noticing. (d) Honest boundaries does not claim to prevent all drift. RELAScripts/hooks/docs-sync-commit-seam.sh (added under BOB-087) now enforces diff sync at the commit seam and passes the explicitly, plus an independent body_md parse oracle. EVIDENCE. docs/qa/BOB-closure-evidence.md, docs/qa/BOB-076/closure-evidence.md, docs/qa/BOB-091/closure-evidence.md. Reported-Via: §11.4.202 reporting direct bug on 2026-08-20T14:46:52Z Reported Claude What (the report, verbatim): merge service on port 7187 wedges after of uptime: the port stays bound and the container keeps reporting “healthy”, but HTTP request hangs until the client times out. Measured 2026-08-20 16:41 UTC+2 on container up 3h53m: curl http://localhost:7187/ -> HTTP 200 in 0.096s (proxy, same process) curl http://localhost:7187/ -> HTTP 000 6.0s (merge service, WEDGED) Both ports served by the SAME process (pid 23300 = 7186, fd 7 = 7187), so this is not a crash worker - one loop inside a live process loop stopped servicing requests while another
BOB-137	Bug	Queued	High	

ATM ID	Type	Status	Severity	Description
				same process is fine. Thread state at the time of the wedge (/proc/2330069/task, 16 threads) - tid 2330529: state R, wchan 0 <- ONE spinning on CPU - tid 2330528: state S, do_sys_poll <- the healthy 7186 poll loop. Other 14: state S, wchan futex_wait_queue. This is the GIL-starvation signature: a thread looping in Python holds the GIL and even other thread queues on the GIL futex. 7187 survives because do_sys_poll releases the GIL. The 7187 async loop needs sustained GIL to service a request and starves. Process was 20.6% while idle. Corroborating evidence: seven sockets held by the process sit in CLOSE-WAIT with unread bytes still in the receive queue (Recv-Q 162, 162, 162, 7187) - clients sent a request and hung up, app never read it or closed the fd. The listener had also accumulated an unaccepted backlog (Recv-Q 6 on the 7187 LISTEN socket) as another observation. The container log's last line before the observation, so the service processed nothing in that window. NOT an exhaustion: only 48 of 16384 fds were open. ROOT CAUSE NOT ESTABLISHED. All loops while True loops in the service (routes.py:search.py:1169, streaming.py:161, streaming.py:359) are correctly bounded and awaited, so the spin is not a naive unslept loop. py-spy could not attach to name the spinning frame - the host has kernel.yama.ptrace_scope=1, which denies non-child attach. Per the §11.4.102 IronPython fix is proposed until the spinning frame is identified. Next diagnostic step: obtain the Python stack. Either run py-spy INSIDE the container (musl wheel), or set ptrace_scope for the duration of one dump, or add a SIGQUIT/fault_handler.register() dump handler to main.py so the running service can be attached for its own stacks without ptrace. DISCUSS CHANNEL (§11.4.238): found by an agent probing the host by hand while investigating an unrelated test-suite result - NOT by automated QA. This is a coverage escape on my own right; see the sibling item on the host check that was structurally incapable of observing it. Affected scope / file-scope manifest: download-proxy/src/main.py, download-proxy/src/merge_service/, download-proxy/src/routes.py, download-proxy/src/search.py, download-proxy/src/streaming.py.

ATM ID	Type	Status	Severity	Description
BOB-139	Bug	Queued	Medium	proxy/src/api/streaming.py, docker-compose.yml (qbittorrent-proxy) Reproduction / context: Leave the qbittorrent-proxy container running for hours with search traffic (a full tests/seed run is sufficient). Then: curl -max-time localhost:7186/ returns 200; curl -max-time http://localhost:7187/ returns 000. Confirmed with: ss -ltnp grep 7187 (unaccepted backlog) and awk '{print \$3}' /proc//task/*/stat (thread, rest futex_wait_queue). Acceptance criteria: The spinning frame is identified as a real Python stack dump (not inferred), the busy-loop is fixed at its root, and a regression guard proves 7187 still answers after a sustained-traffic soak. Evidence: before and after curl timings on both ports plus a thread census showing no permanently-R threads. Reported-Via: §11.4.202 reporting direct bug on 2026-08-20T14:47:35Z Reported-by: Claude What (the report, verbatim): SSE generators in download-proxy/src/api/streaming.py (lines ~153 and ~351) guard their while True loop with this helper: as _client_gone() -> bool: if request is None: return False try: return await request.is_disconnected() except Exception: return False The bare except Exception: False means: if the disconnect check itself raises, the generator concludes the client is STILL CONNECTED and keeps streaming forever. That is fail-OPEN on the only code path that terminates the loop, in a code path that holds a socket and a task. This is the §11.4.201(6) fail-closed rule inverted: the safe default cannot determine whether the client is gone, so to treat it as gone and stop streaming (though the client can always reconnect - SSE is designed for that), not to keep an unbounded stream alive on an unresolvable signal. It is also §11.4.201(6) false-null: a raising probe of a genuinely-connected client return the idempotent False, so the loop cannot distinguish “client here” from “I am blind”. Corroborating observation (2026-08-20): seven sockets held by the merge-service process were sitting in CLOSE-WAIT with unread request bytes, indicating clients had hung up and the server had not noticed. That is consistent with (though not proven to be caused by) this fail-open, a

ATM ID	Type	Status	Severity	Description
BOB-140	Task	Queued	Medium	the reason this is filed rather than left as a style note. HONEST BOUNDARY: not possible to be the cause of the 7187 wedge - both the DO call await asyncio.sleep(poll_interval) they yield and would not busy-spin. This is as a real defect on its own merits (an unbounded stream on an unresolvable stream plus a leaked socket and task), not as the wedge's root cause. Affected scope / fix scope manifest: download-proxy/src/asyncio/streaming.py (_client_gone at ~line 153 ~line 351) Reproduction / context: Monkeypatch Request.is_disconnected to open an SSE stream, disconnect the client, observe the generator never terminates (the loop keeps yielding and the task is never reclaimed). Acceptance criteria: A raise_disconnect probe terminates the stream (closed per §11.4.252) rather than continuing AND a normally-connected client still streams uninterrupted (§11.4.201 both directions). Guard: a unit test for each SSE generator covering raise -> terminate and connect/continue. Reported-Via: §11.4.202 reporting direct task on 2026-08-20T14:56:38Z Reported by: Claude What (the report, verbatim): pre_build/ check_cm_healthcheck_covers_served_ports (landed with BOB-138) implements a rule that every project under this constitution needs a container health check MUST cover every service it serves, because a check probe on a subset asserts a proxy signal instead of a condition (§11.4.201). Its detection logic already carries ZERO boba literals - both the compose file and the served-port manifest have inputs, defaulted from the project root. This therefore belongs in constitution/scripts/healthcheck and should be consumed BY REFERENCE. The thin boba delegator holding only boba's DATA, exactly as check_cm_killpg_pgid_guard.sh was restructured (§11.4.177 / §11.4.28 / §11.4.202). WHY IT WAS NOT UPSTREAMED IMMEDIATELY: a concurrent agent was working in constitution/scripts/gates/ at the time (upstreaming the killpg engine). Two weeks later, one submodule directory risks losing work, so this was deferred deliberately and filed

ATM ID	Type	Status	Severity	Description
BOB-141	Task	Queued	Low	than silently skipped (§11.4.197). This is a known-and-tracked deviation from §11.4.201(6) not an oversight. Two properties MUST be satisfied for the move: 1. FAIL when zero services were checked – a quiet zero from a blind instance is indistinguishable from a clean tree (§11.4.201(6)). 2. FAIL when python3+P is unavailable, rather than skipping, for the same reason. Acceptance: the engine lives in constitution/scripts/gates/, boba’s copy is a thin delegator carrying only its manifest; both directions still verified (a service not a served port FAILs; a fully-covered set of PASSes), and no detection logic is duplicated between the two. Affected scope / file manifest: scripts/pre_build/check_cm_healthcheck_covers_served_ports/config/served_ports.yaml, constitution/scripts/gates/ Reproduction / context: n/a — deviation recorded at landing time, not a discovered defect. Acceptance criteria: Detection engine in constitution/scripts/gates/ boba ships a thin delegator with scope 1 only; zero duplicated detection logic; both polarity directions still verified after the move. Reported-Via: §11.4.202 reporting direct task on 2026-08-20T14:56:38Z Reported by: Claude What (the report, verbatim): CLAUDE.md’s Architecture section states “qbittorrent-proxy-go (Go/Gin, opt-in via -profile go) – replaces the Python proxy 7186, 7187, 7188” The container cannot deliver that. Read from source rather than prose: qBitTorrent-go/Dockerfile:16 CMD ["app/qbittorrent-proxy"] (ONE binary) qBitTorrent-go/Dockerfile:15 EXPOSE 7186 7187 7188 (declares 2) cmd/qbittorrent-proxy/main.go r.Run(fmt.Sprintf(":%d", cfg.ServerPort)) (binds 1) internal/config.go:58 ServerPort = MERGE_SERVICE_PORT (default 7187) qbittorrent-proxy-go container binds 7186 ONLY. webui-bridge is a SEPARATE binary (cmd/webui-bridge, /bridge/health, cfg.BridgePort) that this container never binds and nothing in it binds 7186 either – although the compose service does set PROXY_PORT=7186 and BRIDGE_PORT=7187 in its environment, which reinforces the impression. EXPOSE likewise declares a

ATM ID	Type	Status	Severity	Description
				nothing binds. WHY THIS MATTERS BE TIDINESS: this prose was used as the s of truth when first authoring config/ served_ports.yaml, producing a manifes of [7186, 7187, 7188] for that service. T healthcheck gate built on it then FAILE service whose healthcheck was already - a §11.4.201(1) false-positive refusal ca directly by trusting the doc over the Do The manifest was corrected against sou doc was not, and will mislead the next r the same way. Either the doc is wrong, o Go container is under-provisioned relat intent (it should also run webui-bridge a 7186 listener). Determining WHICH is p this task - do not simply reword the doc match the current binary if the intende was a three-port container. Related: the service's compose block sets PROXY_PO BRIDGE_PORT that no process in the co consumes, and EXPOSE lists 7188 unboc Whichever way the above resolves, thos should agree with reality afterwards. A scope / file-scope manifest: CLAUDE (Architecture + Port Map), docker- compose.yml (qbittorrent-proxy-go env/ EXPOSE), qBitTorrent-go/Dockerfile Reproduction / context: Read qBitTor Dockerfile:16 (single CMD) against CLAUDE.md's 'replaces the Python pro 7186, 7187, 7188'; confirm cfg.ServerPo resolves to MERGE_SERVICE_PORT=7 internal/config/config.go:58. Acceptanc criteria: Doc and container agree, with direction of the fix decided deliberately (correct the doc, or provision the contain match the documented intent). PROXY_ BRIDGE_PORT env and EXPOSE lines a with what the container actually binds.